Leveraging Machine Learning for Long-only Trading: A Tesla Case Study

Abstract: This paper investigates the value of modern machine-learning methods for single-stock return prediction using Tesla as a case study. Building on a large panel of macro, market, and AI-related features, the analysis employs a strictly time-series-consistent design that combines walk-forward cross-validation, Bayesian hyperparameter optimization, and realistic transaction-cost modelling to evaluate a broad set of classifiers and feature-selection schemes. The results show that a Bagging ensemble trained on predictors selected via SLASSO delivers the most attractive risk-adjusted performance, with substantially 192.13% cumulative returns, 1.35 Sharpe ratios, and -29.73% maximum drawdowns. Further evidence indicates that explicitly controlling the temporal stability of selected predictors is crucial for mitigating overfitting and unlocking the benefits of flexible nonlinear learners. Overall, the study highlights both the potential and the limitations of machine learning in empirical asset pricing and provides a transparent experimental template that can be extended to multi-asset portfolios and richer information sets.

Key words: Machine Learning, SLASSO, Bagging, Asset Pricing

1. Introduction

Machine learning has rapidly become a core component of modern financial research, reshaping how economists model, forecast, and interpret asset returns. Its prevalence comes from the ability to process large-scale data, model nonlinear relationships, and adapt to high-dimensional information environments. Compared with traditional econometric approaches, machine learning methods are often more suitable to settings characterized by structural change, interaction effects, and evolving market dynamics (Gu et al., 2020; Amini et al., 2021; Brogaard and Zareei, 2023; Daul et al., 2022). Consequently, these techniques have been widely applied to stock return prediction,

risk management, portfolio allocation, and algorithmic trading.

Within this broader context, Tesla, Inc. provides a particularly relevant and challenging case study. As one of the most closely followed and volatile large-cap equities in the U.S. market, Tesla's valuation depends on a wider set of drivers than many conventional firms. In addition to company-specific fundamentals such as deliveries, margins, and profitability, Tesla's stock price is highly sensitive to macroeconomic conditions, investor sentiment, and expectations regarding future technology-related businesses. These characteristics make Tesla an informative setting in which to examine whether machine learning can improve predictive performance and trading outcomes in a rapidly changing information environment.

Macroeconomic conditions are highly relevant to Tesla because changes in the wider economy can influence both the company's business operations and stock evaluation (Chen et al., 2024). For example, persistent inflation may squeeze household budgets and weaken demand for discretionary purchases such as vehicles. Higher interest rates may create headwinds for growth stocks, since future cash flows are discounted more heavily when rates rise. This effect is particularly important for Tesla, as the company's share price often depends on expectations of substantial long-term growth. Geopolitical tensions add another layer of uncertainty by affecting commodity and energy markets, disrupting supply chains, and raising the cost of key materials like lithium.

Beyond macroeconomic influences, Tesla is frequently valued not merely as an automobile manufacturer, but as a hybrid asset positioned at the intersection of electric vehicles, artificial intelligence, and platform-based technology. Contemporary sell-side research reflects this perspective. Barclays has described Tesla as "passing the baton from auto to AI," while Bank of America emphasizes the option value associated with autonomy, Robotaxi platforms, Full Self-Driving monetization, Optimus humanoid robotics, and the Energy segment. HSBC similarly argues that Tesla's valuation is closely tied to long-term technological vision rather than near-term execution alone. Motivated by this distinctive pricing structure, our research incorporates a novel AI narrative factor designed to capture forward-looking sentiment surrounding Tesla's future AI-related businesses.

Taken together, these considerations suggest that Tesla's return dynamics are shaped by the interaction of macroeconomic forces, firm fundamentals and AI narrative factor. Many of these factors are likely to be nonlinear and time-varying in nature, which makes Tesla a particularly suitable setting for applying machine learning methods.

Despite the growing literature on machine learning in finance, most existing studies focus on forecasting expected returns across broad sets of assets or improving portfolio allocation decisions in diversified investment settings (Gu et al., 2020; Ban et al., 2018; Pinelis and Ruppert, 2022). In such frameworks, model performance is often enhanced by cross-sectional information and risk diversification. By contrast, relatively less attention has been paid to whether machine learning remains effective in a constrained single-asset trading context, where investors cannot rely on diversification and must generate value from timing decisions within one security alone. This question is economically important, as many real-world investors hold concentrated positions in a small number of prominent stocks rather than managing broad portfolios. Within this setting, a long-only strategy is particularly relevant. In practice, many investors are unable or unwilling to short sell because of regulatory restrictions, mandate constraints, borrowing costs, or asymmetric downside risk. These considerations are especially salient for Tesla, whose high volatility and recurring short-squeeze episodes can make short positions costly and difficult to sustain. A long-only framework therefore provides a more demanding and economically meaningful test of model usefulness: rather than profiting from both upward and downward price movements, the strategy must create value by correctly identifying favorable entry opportunities and reducing exposure during adverse periods.

This study develops a machine-learning-based trading framework for Tesla stock that integrates high-dimensional feature engineering, penalized feature selection, time-series-safe model validation, and out-of-sample portfolio evaluation. Specifically, the framework predicts the direction of next-period TSLA returns using a broad set of macroeconomic, market, technical, and AI-related alternative signals, then maps model-implied probabilities into trading positions under a long-flat rule. To mitigate overfitting in a high-dimensional setting, the analysis compares LASSO, stable LASSO

(SLASSO), and Ridge-based screening procedures, and ultimately adopts SLASSO to retain a parsimonious yet robust predictor set that combines conventional financial variables with AI-exposure indicators. Model selection is then conducted across multiple nonlinear classifiers, including random forests, gradient boosting, bagging, multilayer perceptrons, support vector machines, and stacking ensembles, using walk-forward cross-validation and, for selected models, Bayesian hyperparameter optimization. The empirical results show that although several model families generate economically meaningful predictive performance, the SLASSO-plus-bagging specification offers the most attractive overall trade-off between return, Sharpe ratio, turnover, and robustness in the out-of-sample test period.

2. Literature Review

2.1 Machine Learning in Asset Pricing

Traditional empirical asset pricing has long been dominated by linear factor models, in which expected returns are explained by exposure to a limited set of systematic risk factors (Fama and French, 1993; Carhart 1997; Fama and French, 2015). While this framework has generated influential benchmarks, its explanatory power becomes increasingly constrained in modern financial environments characterized by a rapidly expanding "factor zoo," high-dimensional predictor sets, and complex nonlinear interactions among firm characteristics and macroeconomic variables (Harvey et al., 2016; Cochrane, 2011). In such settings, conventional linear models may be vulnerable to misspecification, unstable predictor selection, and weak out-of-sample predictive performance (Kozak et al., 2020; Feng et al., 2020; McLean and Pontiff, 2016). In response to these limitations, machine learning has become an important extension of empirical asset pricing research, as it can efficiently handle large-scale datasets and capture complex functional relationships with fewer restrictive parametric assumptions.

An expanding literature highlights the superiority of machine learning methods over traditional approaches in return prediction and portfolio construction. Gu, Kelly, and Xiu (2020), for example, provide a comprehensive comparison of machine learning algorithms in the cross-section of stock returns and find that tree-based models and neural networks consistently deliver substantial improvements in predictive

performance and economic value. Their findings suggest that much of this gain stems from the models' capacity to exploit nonlinear patterns and interaction effects omitted by standard linear specifications. Subsequent studies further extend the role of machine learning beyond return forecasting to broader asset pricing tasks, including the estimation of risk premia, latent factor discovery, and stochastic discount factor (SDF) construction (Lettau and Pelger, 2020; Chen et al., 2021). Therefore, we adopt machine learning for return prediction due to its ability to effectively process high-dimensional information and capture complex nonlinear patterns and interaction effects, enhancing both predictive accuracy and economic value.

2.2 Regularization and Feature Selection in High-Dimensional Finance

In high-dimensional financial prediction problems, a central challenge lies in separating economically meaningful signals from a large set of noisy and highly correlated predictors. In response, the literature has widely adopted regularization methods to improve estimation accuracy and out-of-sample performance. Among the most influential approaches, LASSO performs simultaneous shrinkage and variable selection by setting weak coefficients exactly to zero, thereby generating sparse and interpretable models. Ridge regression, by contrast, retains all predictors but stabilizes estimation through L2 shrinkage, making it particularly useful under severe multicollinearity. More recent extensions such as Stable LASSO incorporate resampling-based stability selection, retaining only variables that are consistently selected across repeated subsamples (Meinshausen and Bühlmann, 2010; Shah and Samworth, 2013). This addresses the issues of convergence rate and asymptotic consistency of the LASSO algorithm when the number of candidate predictors far exceeds the sample size and there are a large number of non-stationary unit roots (Mei and Shi, 2024). Thus reducing sensitivity to sample-specific noise and improves the robustness of feature selection in financial applications where predictor relevance may vary over time. For these reasons, our research adopts these approaches for regularization and feature selection.

2.3 Nonlinear and Ensemble Models for Return Forecasting

2.3.1 From Linear to Nonlinear: Capturing the Complexity of Financial Markets

Traditional linear models (such as OLS) assume that there is a stable linear relationship between variables, but in high-dimensional financial data, this assumption often fails because it ignores complex nonlinear interaction terms. Gu, Kelly, and Xiu (2020) demonstrated through large-scale empirical comparisons that tree models and neural networks significantly outperform linear benchmarks in prediction because they can effectively characterize high-order interactions. Empirical studies increasingly document that tree-based models and neural networks often outperform traditional linear specifications in forecasting returns and market movements (Fischer and Krauss, 2018; Krauss et al., 2017). These gains are particularly relevant in equity markets, where predictive relationships may involve threshold effects, heterogeneous responses, and regime shifts that cannot be adequately captured by linear structures. Tree-based models such as Random Forest and Gradient Boosting are especially attractive because they can flexibly model such complexities while combining many weak predictors into stronger forecasting signals (Breiman, 2001; Friedman, 2001). Ensemble learning further strengthens predictive performance through complementary mechanisms: Bagging reduces variance through averaging across multiple learners, Boosting improves accuracy through sequential fitting of residual errors, and Stacking combines heterogeneous base models through a meta-learner that integrates diverse predictive signals (Breiman, 1996; Friedman, 2001; Wolpert, 1992). When implemented under a time-series-safe design that preserves the chronological structure of the data, these methods are particularly suitable for financial backtesting and practical trading applications. Chen, Pelger, and Zhu (2024) used deep neural networks to construct an asset pricing model for individual stock returns that fully utilizes a large amount of relevant information and maintains a completely flexible form. Subsequent research (Drobetz and Otto, 2021) further demonstrated that machine learning methods that consider nonlinear relationships and variable interactions can significantly improve earnings prediction capabilities compared to linear baseline models. The latest research has further found that there is a significant nonlinear relationship between asset returns and explanatory variables, and machine learning methods can effectively identify these structures and bring about significant statistical and economic improvements

(Moskowitz et al., 2025).

This non-linearity is particularly evident in narrative-driven assets like Tesla. For example, market sentiment often exhibits Threshold Effects on prices (Nagel, 2021). That is, only when emotions reach a certain critical point will fundamental signals be repriced by the market. This dynamic interaction logic cannot be captured by traditional linear models.

2.3.2 Economic advantages of tree models

Random Forests (Breiman, L., 2001) and Gradient Boosting (Friedman, J. H., 2001) based on recursive partitioning can automatically capture nonlinear relationships and interaction effects between variables. Unlike parametric models, tree models can automatically discover conditional dependencies between variables (Athey & Imbens, 2016). Tree models are naturally robust to outliers (Leippold et al., 2022). For Tesla, which often experiences extreme price fluctuations, this non-parametric feature can effectively prevent a single observation from excessively pulling on the prediction curve, thus maintaining more robust predictive ability during Regime Switching.

2.3.2.1 Random Forest

Random Forests, introduced by Breiman (2001), are ensembles of B decorrelated decision trees. Each tree $h_b(\mathbf{x}; \Theta_b)$ is trained on a bootstrap resample of the training data, and at each split only a random subset of $m \leq p$ features is considered. The final class-probability estimate is:

$$\hat{p}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{p}_b\,(\mathbf{x}; \Theta_b) \qquad (1)$$

where $\hat{p}_b$ is the leaf-fraction estimate of the positive class in tree b. Generalization error is bounded by the strength–correlation trade-off: lower inter-tree correlation and higher individual accuracy reduce ensemble error. In this study, RF is tuned over a grid of {n_estimators, max_depth, min_samples_leaf} via walk-forward cross-validation with best configuration: n_estimators=800, max_depth=6, min_samples_leaf=30.

2.3.2.2 Gradient Boosting Decision Tree

Following Friedman (2001), GBDT fits an additive model $F_M(\mathbf{x}) = \sum_{m=0}^{M} \nu\, h_m(\mathbf{x}; a_m)$ by iteratively minimizing the expected loss. At each stage m, a

regression tree h_m is fitted to the negative gradient (pseudo-residuals) of the loss with respect to the current prediction:

$$\tilde{r}_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}} \qquad (2)$$

For binary classification, the loss is the binomial log-likelihood. The learning rate $\nu \in (0,1]$ controls the contribution of each tree and acts as a regularizer. In this study, GBDT is optimized with Bayesian TPE over {n_estimators ∈ [150, 600], learning_rate ∈ [0.01, 0.2], max_depth ∈ {2,3,4}, subsample ∈ [0.6, 1.0]}, yielding best parameters: n_estimators=600, learning_rate≈0.079, max_depth=2, subsample≈0.916.

2.3.2.3 Bagging Classifier

Bagging (Bootstrap Aggregating), proposed by Breiman (1996), draws B bootstrap samples $\mathcal{D}_b$ of size n' from the training set and trains an independent base learner on each. For classification, the final prediction is a plurality vote (or probability average):

$$\hat{p}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{p}_b(\mathbf{x}) \qquad (3)$$

Bagging reduces variance without significantly increasing bias, making it especially effective for high-variance base learners such as deep decision trees. Here the base learner is a DecisionTreeClassifier, with Bayesian optimization over {n_estimators ∈ [100, 500], max_samples ∈ [0.5, 1.0], max_features ∈ [0.4, 1.0], tree_max_depth ∈ {None, 3, 5, 8}, tree_min_samples_leaf ∈ [5, 60]}, yielding best: n_estimators=100, max_samples≈0.611, max_features≈0.688, tree_max_depth=None, tree_min_samples_leaf=30.

2.3.2.4 Multi-Layer Perceptron

The MLP classifier is a feed-forward neural network with L hidden layers, where each layer applies an affine transformation followed by a non-linear activation $\sigma(\cdot)$:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), l = 1, \ldots, L \qquad (4)$$

The output layer uses a sigmoid function to produce $\hat{p} = \sigma(z^{(L+1)})$. The network

is trained by minimizing binary cross-entropy via stochastic gradient descent with the Adam optimizer. Regularization is provided by L2 weight decay (parameter alpha). Bayesian optimization searches over architecture {hidden_layer_sizes: one of (32,), (64,), …, (256,128,64,32)}, alpha ∈ [1e-6, 1e-2], learning_rate_init ∈ [1e-4, 5e-2], batch_size ∈ {32,64,128,256}, and activation ∈ {relu, tanh}, yielding best: architecture (128,64,32), alpha≈1.08e-6, learning_rate_init≈0.012, batch_size=32, activation=tanh.

2.3.2.5 Support Vector Machine

SVM finds the maximum-margin hyperplane in a kernel-induced feature space. For a kernel $k(\mathbf{x}_i, \mathbf{x}_j)$, the decision function is:

$$f(\mathbf{x}) = \operatorname{sgn}\left(\sum_{i \in \mathrm{SV}} \alpha_i \, y_i \, k(\mathbf{x}_i, \mathbf{x}) + b\right) \qquad (5)$$

Probability estimates are obtained via Platt scaling. The regularization parameter C controls the bias–variance trade-off. A grid over {RBF kernel with $C \in \{1.0,3.0\}$, linear kernel with $C = 1.0$} is searched by walk-forward CV, selecting kernel=linear, C=1.0.

2.3.3 The Financial Logic of Ensemble Learning

The core of ensemble learning lies in constructing a strong learner by combining multiple weak learners. This has profound economic implications in the context of financial forecasting, namely, "hedging prediction errors through diversified decision paths." In individual stock forecasting, the most significant empirical obstacle is the extremely low signal-to-noise ratio. A single model is highly susceptible to capturing random noise within the sample rather than robust economic signals.

Kelly, Pruitt, and Su (2019) pointed out that in low signal-to-noise ratio environments, a single feature often has great estimation uncertainty. Algorithms such as Random Forest, based on Bagging logic, achieve risk diversification at the prediction level through Bootstrap resampling and random feature selection. This mechanism is essentially performing a variance reduction strategy. As discussed by Breiman (1996) and Friedman (2001), ensemble models can effectively reduce model uncertainty by offsetting the estimation bias of different base models at specific sample points, and enhance the robustness of the model by combining multiple prediction paths, thus

exhibiting higher empirical robustness in individual stock return prediction than a single complex model.

Excess returns (Alpha) in financial markets are typically asymmetric and fleeting, and are often hidden within the residuals of linear regression. The sequential learning framework provided by the Boosting algorithm is essentially a relay search of the residuals. Wang and Lera (2025), in their study of return prediction under cross-market structural changes, found that the performance of machine learning models in the face of different market states depends significantly on their ability to adapt to state changes. Boosting can focus the model's attention on "unpredictable observations" by continuously adjusting weights. Furthermore, Chen and Guestrin (2016) pointed out that Boosting has excellent predictive performance in high-dimensional environments, making it an important tool for processing complex financial data. By continuously optimizing the loss function and focusing on "hard samples" where preceding models fail to predict, Boosting can accurately extract economically significant but weakly represented nonlinear signals from a high-dimensional feature set. This iterative optimization capability makes it more advantageous in capturing the premium of assets like Tesla, which is driven by extreme narratives.

Stacking represents an evolution from "predictive averaging" to "predictive learning," and is particularly suitable for dealing with the temporal variability in financial forecasting. The research of Rapach, Strauss, and Zhou (2010) demonstrates that, compared to a single optimal forecasting model, combined forecasting has stronger stability in out-of-sample forecasting by utilizing the heterogeneous information captured by different models. Timmermann (2006) further elaborated theoretically that predictive combination is an effective means of dealing with model structural breakage. Stacking's meta-learner does not simply add weights together, but constructs a conditionally optimal decision model. As discussed by Diebold and Shin (2019), this meta-learning architecture allows the model to learn the historical performance of the base learner under different macroeconomic state variables, thereby enabling dynamic weight allocation. This adaptability provides an end-to-end solution to the complex mechanism switching problem in financial markets, simulating the prudent logic of top

asset management institutions dynamically adjusting strategy weights under different market environments.

2.3.4 Practice of Time-Series-Safe Design in Stacking

To ensure that the empirical results are not contaminated by data mining, this paper strictly adheres to the rigor of financial econometrics during model training. Traditional K-fold cross-validation introduces serious look-ahead bias when dealing with financial time series with serial correlation (Dixon et al., 2020). Following the Purged Validation principle proposed by Lopez de Prado (2018), an extended window is used for out-of-sample evaluation. This design ensures that the meta-learner assigns weights based solely on historical information. Harvey, Liu, and Zhu (2016), in their statistical significance framework for factor pricing research, pointed out that traditional significance levels may lead to false positives due to massive testing. They proposed using a more stringent statistical threshold to control the risk of false discovery, thereby improving the robustness and reliability of the results. This aligns with our emphasis on the rigor of statistical evaluation in meta-learner weight assessment, thus ensuring that the prediction results have a certain degree of robustness in the market (Hou, Xue, and Zhang, 2017). Stacking builds a meta-learner on top of base-learner predictions. To avoid lookahead bias in time-series data, a custom TimeSeriesStackingClassifier is implemented using TimeSeriesSplit for out-of-fold (OOF) meta-feature generation. Let $\hat{p}_j^{(\mathrm{OOF})}$ be the OOF probability from base learner j. The meta-feature matrix is:

$$\mathbf{Z} = \left[\hat{p}_1^{(\mathrm{OOF})}, \hat{p}_2^{(\mathrm{OOF})}, \dots, \hat{p}_J^{(\mathrm{OOF})}\right] \qquad (6)$$

The meta-learner (Logistic Regression with $C \in [0.1,10]$) is fitted on $\mathbf{Z}$. Base learners are RF and GBDT. Bayesian optimization over 12 trials searches across their joint hyperparameter space plus meta_c and passthrough.

2.4 Textual Narratives and AI-Driven Asset Pricing

2.4.1 Incremental value of textual information and narrative pricing

In traditional asset pricing frameworks, factor construction is mostly limited to low-frequency financial statements or high-frequency price and volume data. However, in recent years, many authoritative scholars, such as Gentzkow et al. (2019), have

introduced text as an input method for economic research, systematically demonstrating the legitimacy of text as an economic variable. They argue that textual data contains 'soft information' that financial indicators cannot capture, forming effective incremental information about market fundamentals. Text is not noise, but contains incremental information beyond financial statements, and narratives can be priced like factors. Loughran and McDonald (2011) pioneered the standardization of financial texts by developing a glossary that links indicators such as 10-K filings, trading volume, return volatility, fraud, material defects, and unexpected gains. Shiller's (2017) narrative economics framework provides solid theoretical support for the "narrative-driven price fluctuations" theory, stating that "narratives can spread rapidly and widely, even globally, thereby generating economic impact." Jegadeesh et al. (2013) demonstrated that the sentiment in the text has a strong predictive power for market reactions, and that such reactions cannot be explained by simple quantitative indicators.

A growing body of research indicates that news reports, analyst reports, and social media texts contain incremental information beyond quantitative variables. This type of text not only reflects investors' heterogeneous beliefs in real time but also predicts returns, volatility, and risk premiums, and captures the evolution of market expectations regarding future uncertainty. Manela et al. (2017) developed a text-based measure of uncertainty starting from 1890. Bybee et al. (2020) empirically demonstrated that text-based inputs can accurately track a wide range of economic activity indicators and have incremental predictive power in forecasting macroeconomic outcomes, surpassing standard numerical predictive indicators. Studies like Narrative Asset Pricing further demonstrate that textual narratives themselves can constitute interpretable systematic risk factors, which exhibit out-of-sample robustness in explaining long-term returns and uncertainty premiums compared to traditional price-volume models (Da et al., 2011).

2.4.2 From Lexical Approach to FinBERT: The Evolution of Financial Semantic Understanding

Regarding texts on specific topics, Engle et al. (2020) constructed a hedging factor based on news texts. This demonstrates that textual narratives on specific topics can be extracted as asset pricing factors. However, the choice of tools is crucial for accurately

extracting Tesla's AI narrative from massive amounts of text. Financial texts are highly specialized, context-dependent, and asymmetric. Text sentiment extraction technology has undergone a paradigm shift from dictionary-based to deep learning. Early dictionary methods, such as Tetlock's (2007) quantitative research method on the interaction between media and the stock market, etc., are difficult to handle in terms of complex semantic dependencies in financial contexts, as pointed out by Hoberg and Phillips (2016). They struggle to handle the special meanings within financial contexts.

BERT is designed to pre-train deep bidirectional representations on unannotated text by considering both left-hand and right-hand contexts across all layers. FinBERT (Araci, 2019; Yang et al., 2020) is introduced, a domain-specific language model pre-trained on a large-scale financial corpus. With breakthroughs in the Transformer architecture (Devlin et al., 2018), FinBERT achieves deep semantic understanding of finance through pre-training on ultra-large-scale financial corpora (such as SEC filings and Analyst reports). In the context of asset pricing, this technological leap means we can more accurately capture marginal sentiment shifts regarding "AI transformation" or "breakthroughs in autonomous driving technology." The accuracy of this sentiment extraction directly determines the signal-to-noise ratio of the Narrative Factor. Compared to traditional methods, FinBERT effectively eliminates ambiguity in financial terminology. Huang et al. (2023) found that FinBERT's advantage over other algorithms and Google's original bidirectional encoder representation model is particularly pronounced when the training sample size is small and contains financial terms that are not commonly found in general text. FinBERT also outperforms other models in recognizing discussions related to environmental, social, and governance issues. This provides a more rigorous technical foundation for accurately extracting the AI narrative of cross-industry, highly complex companies like Tesla.

2.4.3 Literature Gap and the Innovation of This Paper -- The Research Gap: Firm-Specific AI Narrative

While significant progress has been made in textual asset pricing, existing literature largely focuses on broad market sentiment or macro-level policy uncertainty. The Policy Uncertainty Index (EPU), developed by researchers such as Baker, Bloom, and

Davis (2016), demonstrates the powerful explanatory power of macro narratives on asset prices. However, these "general-purpose" indices often obscure the heterogeneous responses of different industries and firms to specific technological shocks. As inspired by Giglio, Kelly, and Liao (2021) in their discussion of systemic risk, asset pricing needs to consider not only macroeconomic fluctuations but also "dimensional selection" for specific dimensions of risk. Existing general sentiment indicators are often too coarse to characterize the pricing effects of disruptive technologies like AI on specific firms.

While academia has begun to focus on textual information at the individual stock level, research depth remains limited to simple "bullish/bearish" sentiment. Early individual stock studies (Antweiler and Frank, 2004) confirmed that internet discussions are not simply noise, but they were limited by simple classifiers such as Naive Bayes, making it difficult to capture deep semantics. Cookson and Niessner (2020) found evidence that users selectively access supporting information, demonstrating the central role of investor belief heterogeneity in individual stock pricing, which provides valuable support for individual stock textual analysis.

This paper constructs a high-frequency AI narrative factor for Tesla, leveraging its "technology + manufacturing" attributes, by combining AI keyword filtering and the FinBERT model. The innovation lies in integrating AI keyword filtering, FinBERT sentiment, AI intensity, and sentiment to create a thematic narrative factor specific to Tesla. Currently, very little literature delves into the dynamic relationship between thematic narratives and individual stock valuations. Technological innovation plays a crucial role in the medium-term fluctuations of overall economic growth and total factor productivity; when companies engage in cutting-edge technological innovation, their valuations undergo structural restructuring. Kogan et al. (2017) applied patent data obtained by US companies and combined it with stock market reactions to patent announcements, finding a positive correlation between the private economic value estimation at the patent level and the scientific value of these patents. López-Lira and Tang (2025) also proves the predictive power of large language models (LLMs) like ChatGPT is stronger than traditional sentiment analysis methods.

2.5 Time series validation, out-of-sample testing, and over-fitting control

While machine learning's high fitting ability improves prediction accuracy, it also significantly increases the risk of over-fitting and data snooping. In noisy data such as individual stock returns, models are highly susceptible to capturing non-reproducible spurious signals. Sullivan, Timmermann, and White (1999) highlighted "data mining bias" caused by repeatedly searching historical data. Chordia, Goyal, and Saretto (2020) emphasized that uncontrolled multiple testing can lead to up to 45% false positives when dealing with extremely large sets of strategies or predictor variables, suggesting that simple significance tests may severely overestimate the significance of in-sample performance in large-scale strategy screening. McLean and Pontiff (2016) found that many predictive signals significantly decayed after publication, implying the prevalence of in-sample overfitting.

To address these risks, this paper constructs a rigorous Training-Validation-Test Separation evaluation system in the experimental design. Unlike traditional random partitioning, this paper ensures that the hyperparameter optimization process does not "peep" into the final test data by physically isolating the validation set from aHoldout Test Set. Following Gu, Kelly, and Xiu (2020), the analysis relies on a fully independent holdout test to provide a stringent out-of-sample assessment of economic performance and to mitigate concerns that in-sample fit merely reflects statistical noise or overfitting, a concern emphasized by Hou, Xue, and Zhang (2020).

A core characteristic of financial data is its temporal relevance, requiring the validation process to strictly prohibit any form of future information leakage. Lopez de Prado (2018) systematically criticized the data leakage and forward bias resulting from applying k-fold cross-validation in the financial field. Although Bergmeir and Hyndman (2018) argued that this method has theoretical validity under weakly dependent sequences, this paper insists on using the TimeSeriesSplit architecture for highly volatile assets like Tesla. This architecture ensures that the validation set always follows the training set on the timeline, logically and rigorously simulating the "progressive information set" faced by investors in the real market, thereby avoiding evaluation bias caused by sequence relevance.

Furthermore, Rossi (2013) and Timmermann (2008) emphasized the temporal instability of asset return prediction relationships. To address this challenge, this paper introduces Walk-forward Tuning. By continuously retraining the model and updating the optimal hyper-parameter combination within a rolling window, the model can capture dynamically changing mechanistic features. This design not only ensures the robustness of the prediction process but also guarantees that the AI narrative factor constructed in this paper still possesses reliable out-of-sample pricing capabilities in the face of dramatic structural changes in the market.

3 Research Design

3.3 Data Construction

To determine which factors contribute most substantially to the predictive content of trading signals, we collect 186 candidate factors spanning the sample period from 2016-03-31 to 2026-02-28. Consistent with the preceding discussion, the factor universe integrates macroeconomic variables, firm-level fundamentals, and AI-narrative information, reflecting the premise that Tesla's return-generating process is driven by heterogeneous and potentially interacting information sources rather than by any single economic block.

The macro-financial predictor set spans the yield curve, monetary policy uncertainty, inflation dynamics, real activity, exchange-rate conditions, labor-market fundamentals, and energy- and commodity-related indicators. The underlying data are sourced from Wind.

The firm-level predictor set comprises measures of profitability, solvency, operating efficiency, cash flow, and growth. These variables are constructed from quarterly accounting information reported in Form 10-Q filings and obtained from Compustat, with selected measures, such as year-over-year growth rates, computed directly from the raw accounting items. To mitigate look-ahead bias, the accounting data are aligned to their first public disclosure dates rather than to the fiscal period ends they describe, and each variable is carried forward until updated by a subsequent release. This timing convention ensures that the information set matches what investors could have observed in real time.

3.4 Construction of the AI Narrative Factor

The AI narrative factor is constructed from Tesla-related textual disclosures in four steps. First, a Tesla-specific keyword dictionary is developed to identify text associated with the firm's core AI themes, including autonomous driving, perception algorithms, data and training infrastructure, compute and chips, robotics, and AI commercialization. The dictionary is then used to isolate AI-relevant text from the broader corpus, so that subsequent sentiment measurement is restricted to observations in which AI-related content is salient. For each document i on date t, AI narrative intensity is defined as the frequency of AI-related keywords scaled by document length. Let $\mathcal{K}$ denote the Tesla-specific AI keyword set and let $Text_{i,t}$ denote the text content of document i. The intensity measure is defined as

$$\text{AIIntensity}_{i,t} = \frac{\#\{k \in \mathcal{K} : k \text{ appears in } Text_{i,t}\}}{\text{Length}(Text_{i,t})} \times 1000. \qquad (7)$$

Text length is measured by word count for ASCII text and by character count otherwise, so that the measure remains comparable across documents of different formats. Sentiment is extracted using FinBERT from sentences that contain at least one AI-related keyword. For each AI-relevant sentence j within document i, the sentence-level sentiment score is defined as

$$s_{i,t,j} = \begin{cases} p^{+}_{i,t,j}, & \text{if the predicted label is positive,} \\ -p^{-}_{i,t,j}, & \text{if the predicted label is negative,} \\ 0, & \text{if the predicted label is neutral,} \end{cases} \qquad (8)$$

where $p^{+}_{i,t,j}$ and $p^{-}_{i,t,j}$ denote the predicted probabilities for the positive and negative classes, respectively. The document-level AI sentiment score is then computed as the average across AI-relevant sentences:

$$\text{AISentiment}_{i,t} = \frac{1}{N_{i,t}} \sum_{j=1}^{N_{i,t}} s_{i,t,j}, \qquad (9)$$

where $N_{i,t}$ denotes the number of AI-related sentences in document i. If no AI-related sentence is identified, the sentiment score is set to zero. The document-level AI narrative factor is defined as the interaction between AI narrative intensity and AI sentiment:

$$\text{AINarrativeFactor}_{i,t} = \text{AIIntensity}_{i,t} \times \text{AISentiment}_{i,t}. \quad (10)$$

This construction ensures that the factor increases when AI-related discussion is both more prominent and more positively framed, and decreases when AI narratives are more negative in tone. The event-based implementation used in the main pipeline adopts a more granular weighting scheme. For each AI-related event i on date t, the code first defines a relevance weight as

$$w_{i,t} = \frac{\text{relevance}_{i,t}}{100} \times \frac{\text{event_relevance}_{i,t}}{100}. \quad (11)$$

Let $KC_{i,t}$ denote the number of AI keyword matches in the event text, truncated from below at one once the event is identified as AI-related. Let $S_{i,t}$ denote the FinBERT-based event sentiment score. The event-level factor contribution is then given by

$$\text{FactorEvent}_{i,t} = w_{i,t} \times KC_{i,t} \times S_{i,t}, \quad (12)$$

and the corresponding AI narrative volume is defined as

$$\text{VolumeEvent}_{i,t} = w_{i,t} \times KC_{i,t}. \quad (13)$$

Hence, each event contributes more to the final factor when it is more relevant, contains denser AI-related content, and is associated with stronger sentiment. The daily AI narrative factor is obtained by aggregating all event-level contributions on date t:

$$\text{FactorSum}_t = \sum_{i \in t} \text{FactorEvent}_{i,t}, \quad (14)$$

$$\text{VolumeSum}_t = \sum_{i \in t} \text{VolumeEvent}_{i,t}. \quad (15)$$

Using these quantities, the code also constructs a daily weighted-average sentiment measure,

$$\text{SentimentWAvg}_t = \frac{\text{FactorSum}_t}{\text{VolumeSum}_t}, \text{if } \text{VolumeSum}_t > 0, \quad (16)$$

and sets it to zero otherwise. This measure summarizes the average tone of AI-related discussion after weighting by relevance and keyword intensity. Finally, to capture unusual shifts in Tesla's AI-related information environment, the daily factor is standardized using a rolling window. Specifically, the code computes a 30-day rolling z-score:

$$\text{FactorZ}_t = \frac{\text{FactorSum}_t - \mu_{t,30}}{\sigma_{t,30}}, \qquad (17)$$

where $\mu_{t,30}$ and $\sigma_{t,30}$ denote the rolling mean and rolling standard deviation of FactorSum_t, respectively, subject to a minimum of ten observations. This standardized series is used to identify narrative shocks, that is, periods in which AI-related discussion around Tesla becomes unusually strong relative to its recent history. Taken together, the resulting AI narrative factor captures both the **salience** and the **valence** of Tesla-related AI discussion in text and provides a daily measure of the market's evolving AI narrative around the firm.

3.5 Prediction Targeting and Data Pre-processing

The prediction task is formulated as a one-step-ahead directional forecasting problem for Tesla's daily stock return. Specifically, the raw dataset is first sorted in chronological order, duplicate observations by date are removed, and the target return is defined as the next-day simple return obtained by shifting the contemporaneous return series forward by one period. Let r_t denote Tesla's observed daily return on day t. The forecasting target is the next-period return, defined as

$$r_{t+1}^{\text{target}} = r_{t+1}. \qquad (18)$$

The corresponding binary classification label is constructed as

$$y_t = \mathbf{1}\left(r_{t+1}^{\text{target}} > 0\right), \qquad (19)$$

where $\mathbf{1}(\cdot)$ is the indicator function that equals one when the next-day return is positive and zero otherwise.

This labeling scheme implies that predictors observed at date t are used to forecast the sign of the return realized over $t+1$, thereby preserving the correct forecasting chronology. Observations with missing next-day returns, which arise mechanically at the end of the sample after the forward shift, are removed before model estimation.

The predictor matrix is constructed from the numeric columns of the merged dataset after excluding the explicit target variables. In particular, the feature set removes next_return and y_up, and any column that is entirely missing is dropped from the design matrix. Dates are retained as the index for alignment, backtest reporting, and the construction of next-period trading records.

Formally, let $X_t \in \mathbb{R}^p$ denote the predictor vector available at date t. The final supervised learning sample is therefore given by

$$\mathcal{D} = \{(X_t, y_t, r_{t+1}^{\text{target}})\}_{t=1}^{T-1}. \qquad (20)$$

This representation makes clear that the classification target and the realized economic payoff are linked, but not identical: the model is trained on the sign of next-day returns, while economic performance is evaluated using the realized magnitude of those returns in the backtest. To avoid look-ahead bias, the sample is partitioned strictly by time rather than randomly shuffled. The code uses a chronological split function to divide the sample into training, validation, and test subsets, with default fractions of 60 percent, 20 percent, and 20 percent, respectively. If T denotes the full sample size, the split can be written as

$$\mathcal{D}_{\text{train}} = \{1, \dots, \lfloor 0.6T \rfloor\}, \mathcal{D}_{\text{val}} = \{\lfloor 0.6T \rfloor + 1, \dots, \lfloor 0.8T \rfloor\}, \mathcal{D}_{\text{test}} = \{\lfloor 0.8T \rfloor + 1, \dots, T-1\}. \qquad (15)$$

Because the split is purely forward-looking, the validation and holdout test periods mimic real-time forecasting conditions. The code also applies an additional pre-filter to the AI-derived variables before downstream feature selection. Using training-sample information only, AI features with excessive missingness are removed, and highly collinear AI variables are further screened out through a greedy correlation filter. Let m_j denote the missing rate of feature j in the training set and ρ_{jk} denote the pairwise correlation between features j and k. The retained AI feature set satisfies

$$m_j \leq \tau_m, \qquad (21)$$

and, conditional on prior retention of feature k,

$$| \rho_{jk} | < \tau_\rho, \qquad (22)$$

where the default thresholds are $\tau_m = 0.60$ and $\tau_\rho = 0.95$.

After the train-validation-test split, model estimation is carried out within preprocessing pipelines that standardize the treatment of missing values and scale heterogeneity across predictors. For the classification models, each pipeline applies median imputation followed by z-score standardization before fitting the classifier. Denoting the imputed value of feature $x_{j,t}$ by $\tilde{x}_{j,t}$, the standardized feature is

$$z_{j,t} = \frac{\tilde{x}_{j,t} - \mu_j}{\sigma_j}, \qquad (23)$$

where μ_j and σ_j are computed from the corresponding training sample only.

For the feature-selection stage, the preprocessing is slightly more restrictive. In addition to median imputation and standardization, the code removes zero-variance predictors through a variance-threshold filter before applying LASSO, SLASSO, or Ridge selection. This step ensures that coefficient-based feature selection is not distorted by degenerate predictors with no cross-sectional or time-series variation in the estimation sample. An important implication of this design is that every preprocessing transformation is fit on the training portion of each estimation step and then applied out of sample to the subsequent validation or test observations. As a result, the entire prediction pipeline—including imputation, scaling, feature filtering, and model fitting—remains aligned with a genuine forecasting exercise rather than an in-sample classification problem. This preprocessing design ensures that the empirical framework targets an economically meaningful forecasting object, preserves temporal causality, and provides a clean foundation for subsequent feature selection, hyperparameter tuning, and out-of-sample trading evaluation.

3.6 Signal Generation, Portfolio Construction, and Performance Evaluation

After feature selection, each classifier is trained to estimate the probability that Tesla's next-day return is positive. Let $\hat{p}_t = \Pr(y_t = 1 \mid X_t)$ denote the model-implied probability of an upward move at time t. The forecasting output is therefore probabilistic rather than purely categorical, which allows the trading rule to map prediction confidence into position decisions. The code supports three position-construction schemes: long-short, long-flat, and long-flat-quantile. In the empirical implementation, the default setting is long-flat, meaning that the strategy holds a long position when the estimated probability exceeds a selected threshold and otherwise stays out of the market. Formally, under the long-flat rule, the portfolio position is defined as

$$w_t = \begin{cases} 1, & \hat{p}_t \geq \tau, \\ 0, & \hat{p}_t < \tau, \end{cases} \qquad (24)$$

where τ is the classification threshold. For completeness, the alternative long-short rule used in the framework can be written as

$$w_t = \begin{cases} 1, & \hat{p}_t \geq \tau, \\ -1, & \hat{p}_t < \tau, \end{cases} \qquad (25)$$

which forces the portfolio to remain continuously invested on either the long or short side. The third specification, long-flat-quantile, determines the threshold endogenously from the training-sample probability distribution and enters a long position only when the forecast lies above a specified quantile. If $Q_q(\hat{p}_{\text{train}})$ denotes the q-th quantile of training-sample predicted probabilities, then

$$w_t = \mathbf{1}\big(\hat{p}_t \geq Q_q(\hat{p}_{\text{train}})\big). \qquad (26)$$

Given the trading position w_t, the raw strategy return is computed by multiplying the position by the realized next-day return. Let r_{t+1} denote the realized simple return over the following trading day. The gross strategy payoff is therefore

$$r_{t+1}^{\text{gross}} = w_t \, r_{t+1}. \qquad (27)$$

This formulation links the classification output to an economically interpretable backtest, since the model is rewarded only when the chosen market exposure aligns with the realized next-day return direction and magnitude. To make the backtest more realistic, the strategy deducts linear transaction costs based on portfolio turnover. Let w_{t-1} and w_t denote consecutive portfolio positions. Turnover is defined as the absolute change in exposure,

$$\text{Turnover}_t = | \, w_t - w_{t-1} \, |. \qquad (28)$$

With c denoting the cost per unit of position change, trading cost is calculated as

$$\text{Cost}_t = c \; | \, w_t - w_{t-1} \, |. \qquad (29)$$

Accordingly, the net strategy return becomes

$$r_{t+1}^{\text{net}} = w_t r_{t+1} - c \; | \, w_t - w_{t-1} \, |. \qquad (30)$$

This cost specification has a clear economic interpretation in the code implementation. A change from 0 to 1 incurs one unit of trading cost, whereas a reversal from -1 to 1 incurs two units because the position must first be closed and then reopened in the opposite direction. The first-period position is initialized from zero, which ensures that entry into the market is also charged consistently. Portfolio

performance is then evaluated using the time series of net strategy returns. The cumulative equity curve is computed recursively as

$$\text{Equity}_T = \prod_{t=1}^{T}(1 + r_t^{\text{net}}), \qquad (31)$$

and cumulative return is defined by

$$R^{\text{cum}} = \text{Equity}_T - 1. \qquad (32)$$

To facilitate comparability across models, the code also annualizes strategy performance. If T denotes the number of trading observations and 252 is used as the number of trading days per year, the annualized return is computed as

$$R^{\text{ann}} = (1+R^{\text{cum}})^{252/T} - 1. \qquad (33)$$

Annualized volatility is estimated from the standard deviation of daily net returns,

$$\sigma^{\text{ann}} = \sigma(r_t^{\text{net}})\sqrt{252}. \qquad (34)$$

Based on these quantities, the Sharpe ratio is calculated as

$$\text{Sharpe} = \frac{\bar{r}^{\text{net}}}{\sigma(r_t^{\text{net}})}\sqrt{252}, \qquad (35)$$

where $\bar{r}^{\text{net}}$ is the mean daily net return. Since the implementation does not subtract a separate risk-free rate, this is effectively an excess-return-free Sharpe proxy commonly used in short-horizon strategy backtests. Risk is further assessed through maximum drawdown computed from the strategy equity curve. Let E_t denote cumulative equity up to time t, and let $\max_{s\le t} E_s$ be the running peak. The drawdown at time t is

$$DD_t = \frac{E_t}{\max\limits_{s\le t} E_s} - 1, \qquad (36)$$

and maximum drawdown is given by

$$\text{MDD} = \min_t DD_t. \qquad (37)$$

This measure captures the most severe peak-to-trough loss experienced by the strategy during the evaluation window. Besides economic metrics, the code also reports standard classification measures on the holdout test set. Specifically, test accuracy is computed by thresholding predicted probabilities into class labels, and the area under the ROC curve (AUC) is used to assess ranking quality of the probabilistic forecasts.

These two indicators complement the return-based measures by showing whether statistical classification quality aligns with realized trading profitability. A further strength of the framework is that validation and test performance are separated carefully. Hyperparameters and decision thresholds are chosen on the development sample through walk-forward validation, whereas the final economic results are reported only on the untouched out-of-sample test period. This design ensures that the reported cumulative return, Sharpe ratio, and maximum drawdown are genuine holdout outcomes rather than in-sample optimized statistics. For benchmarking purposes, the code also implements a passive always-long baseline strategy. Under this benchmark, $w_t = 1$ for all periods, so the strategy simply tracks Tesla's directional exposure while still incorporating transaction-cost accounting at the initial entry. Comparing machine-learning models against this passive baseline provides a direct test of whether predictive signals add value beyond unconditional long exposure to the stock.

3.7 Feature Selection

3.7.1 Methodology Overview

Three penalized regression methods are applied to select predictive features from the full feature space, each operating on a standardized and imputed feature matrix via a preprocessing pipeline (median imputation → standard scaling → variance thresholding). The regression target is the next-day simple return r_{t+1}, and alpha selection is performed by time-series cross-validation with $K = 5$ folds (TimeSeriesSplit). All feature selection is performed exclusively on the training set to prevent validation leakage.

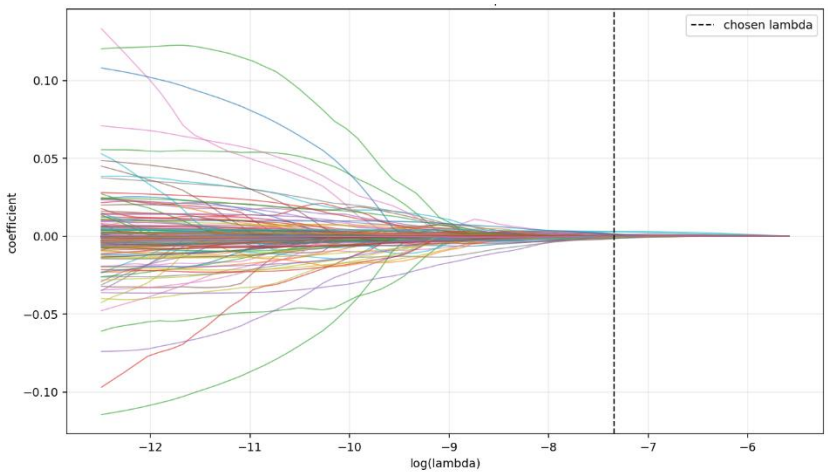


Figure 1 LASSO coefficient path

LASSO (Least Absolute Shrinkage and Selection Operator). Proposed by Tibshirani (1996), LASSO solves:

$$\hat{\beta}_{\text{LASSO}} = \arg\min_{\beta}\left\{\frac{1}{n}\parallel \mathrm{y} - \mathrm{X}\beta \parallel_2^2 + \lambda \parallel \beta \parallel_1\right\} (38)$$

The ℓ_1 penalty promotes exact sparsity: coefficients of irrelevant features are shrunk to exactly zero, yielding a parsimonious feature subset. The optimal λ^* is selected by minimizing cross-validation MSE along a path of 60 penalty values, with a minimum-feature constraint of 30 to prevent over-regularization. The regularization path is visualized in Figure 1.

Ridge Regression. Ridge regression, proposed by Hoerl and Kennard (1970), solves:

$$\hat{\beta}_{\text{Ridge}} = \arg\min_{\beta}\left\{\frac{1}{n}\parallel \mathrm{y} - \mathrm{X}\beta \parallel_2^2 + \lambda \parallel \beta \parallel_2^2\right\} (39)$$

The ℓ_2 penalty shrinks all coefficients toward zero but does not produce exact sparsity. Ridge therefore requires a secondary thresholding rule: features are selected if $\mid \hat{\beta}_j \mid \geq \rho \cdot \max_k \mid \hat{\beta}_k \mid$, where the ratio $\rho = 0.10$ is applied after RidgeCV selects λ^* by cross-validation. Min/max feature count constraints (20–60) are enforced.

Stable LASSO (SLASSO). SLASSO extends standard LASSO with the bootstrap stability selection framework proposed by Meinshausen and Bühlmann (2010). The algorithm proceeds in two stages:

1. Alpha selection: λ^* is chosen by time-series cross-validation with LassoCV (same as LASSO).
2. Bootstrap stability scoring: For $B = 80$ bootstrap resamples of

fraction $f = 0.75$ of the training set, LASSO is re-fitted and a selection frequency $\hat{\Pi}_j$ is computed for each feature:

$$\hat{\Pi}_j = \frac{1}{B}\sum_{b=1}^{B} 1\left[\,|\,\hat{\beta}_j^{(b)}\,|> 0\right] (40)$$

Features with $\hat{\Pi}_j \geq \tau_{\text{stab}} = 0.60$ are retained. This guards against the well-known instability of LASSO, where small data perturbations may produce very different selected sets. A minimum feature constraint of 30 is enforced as a fallback.

3.7.2 Feature Selection Outcomes

Feature selection is performed through three procedures: LASSO, Stable LASSO, and Ridge. LASSO selects sparse predictors through time-series cross-validation, Stable LASSO adds bootstrap-based stability selection, and Ridge relies on coefficient ranking plus explicit lower and upper bounds on retained feature counts because L2 regularization shrinks coefficients without setting them exactly to zero.

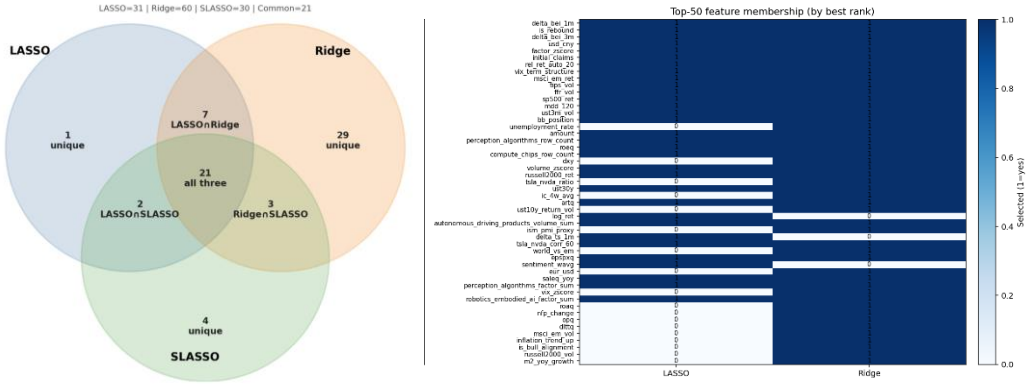


Figure 2 Feature overlap across LASSO/Ridge/SLASSO

LASSO selected 32 features and Ridge selected 60, with an overlap of 29 features (Jaccard similarity = 0.46). Of the 32 LASSO features, 90.6% are also present in the Ridge set, indicating that most LASSO-selected features are verified as having non-trivial Ridge coefficients. Three features are selected exclusively by LASSO (log_ret, delta_ts_1m, sentiment_wavg), and 31 features appear only in the Ridge set (mostly lower-ranked macroeconomic and technical factors). The top jointly selected features (ranked by best rank across LASSO and Ridge) include: delta_bei_1m(breakeven inflation 1-month change), is_rebound(mean-reversion

indicator), factor_zscore (composite factor z-score), initial_claims (US jobless claims), rel_ret_auto_20 (TSLA relative return vs. auto sector), vix_term_structure, mdd_120(maximum drawdown over 120 days), and msci_em_ret. SLASSO's 30 features largely overlap with LASSO's but additionally include several AI-narrative features (data_training_loop_factor_sum_shock_rel, compute_chips_factor_sum) and technical signals (Volume, death_cross, high_inflation_dummy) that are excluded by standard LASSO. While standard Ridge regression does not produce exact zeros in coefficients, our feature selection implementation applies a post-hoc threshold-based filtering layer. Specifically, we retain features with |coefficient| ≥ min_coef_ratio × max(|coefficients|), where min_coef_ratio = 0.10 by default. This converts Ridge into a sparsity-inducing method via coefficient magnitude thresholding, rather than the L1 penalty used by LASSO. The resulting 60-feature set thus represents the features with the largest Ridge coefficients after this thresholding mechanism and min/max feature count constraints.

The high overlap between LASSO and Ridge provides an important robustness signal: features consistently identified by methods with fundamentally different penalty structures are likely to reflect genuine predictive content rather than numerical artifacts. The 29-feature consensus set constitutes a high-confidence core feature set. LASSO is well-suited for high-dimensional settings where true sparsity is assumed. Its automatic zero-shrinkage produces highly interpretable, parsimonious models and is computationally efficient. However, LASSO is known to be selection-unstable: resampling the training data can yield substantially different non-zero sets, particularly when features are correlated. In this experiment, LASSO also exhibits a tendency to exclude informationally related features by arbitrarily zeroing out one of a correlated pair. Ridge does not enforce sparsity and consequently selects a much larger feature set (60 features), including lower-signal macroeconomic variables. Its strength lies in stability under multicollinearity — Ridge coefficients vary smoothly with the data, avoiding the discontinuous zero-selection behavior of LASSO. However, this comes at the cost of reduced parsimony: the broader feature set increases model complexity and may introduce noise. SLASSO addresses LASSO's instability by requiring consistent

selection across bootstrap resamples. This approach, which aligns with the formal stability guarantees of Meinshausen and Bühlmann (2010), yields a more robust feature set than standard LASSO while maintaining sparsity comparable to LASSO. SLASSO's additional AI-narrative features (e.g., compute_chips_factor_sum, data_training_loop_factor_sum_shock_rel) reflect the unique information content of alternative data sources not captured by traditional macro factors. As the final chosen feature selector for downstream modeling, SLASSO provides the best balance between parsimony, stability, and signal breadth.

3.8 Walk-Forward Validation Framework

To avoid data snooping, all hyperparameter selection and threshold tuning is performed using a walk-forward cross-validation scheme. The development set (train + validation) is split into $K = 5$ folds via sklearn.model_selection.TimeSeriesSplit. In each fold k:

1. The model is trained on all data up to fold k.
2. Out-of-sample (OOS) strategy returns are collected for fold $k + 1$.
3. Concatenated OOS returns form the validation performance estimate.

The optimal hyperparameter set θ^* and threshold τ^* are chosen by maximizing the OOS annualized return:

$$\theta^*, \tau^* = \arg\max_{\theta,\tau} \quad Ann\left(\tilde{r}^{OOS}(\theta, \tau)\right) \qquad (41)$$

Final models are retrained on the full development set with (θ^*, τ^*) and evaluated once on the held-out test set.

3.9 Bayesian hyperparameter optimization

Grid search and random search are commonly used for hyperparameter tuning, but both are inefficient when the search space is high-dimensional or when model evaluation is expensive. This study applies Bayesian hyperparameter optimization via the Tree-structured Parzen Estimator algorithm, implemented through the Optuna framework. Bayesian optimization casts hyperparameter search as a sequential decision problem. Given a black-box objective $f: \mathcal{X} \to \mathbb{R}$, the goal is to find:

$$\theta^* = \arg\max_{\theta \in \mathcal{X}} \quad f(\theta) \qquad (42)$$

At each trial t, a *surrogate model* $\hat{f}$ is fitted to all prior observations $\mathcal{D}_{t-1} = \{(\theta_i, y_i)\}_{i<t}$, and an *acquisition function* $\alpha(\theta; \mathcal{D}_{t-1})$ is maximized to propose the next candidate. The general SMBO loop is:

$$\theta_t = \arg\max_{\theta} \quad \alpha(\theta; \mathcal{D}_{t-1}) \qquad (43)$$

This exploits past evaluations to propose better candidates, dramatically outperforming random search in both sample efficiency and final performance quality (Bergstra et al., 2011). The TPE algorithm, introduced by Bergstra et al. (2011) and further analyzed in the context of the Optuna framework by Akiba et al. (2019), models the surrogate using two separate density estimates rather than a direct $P(y \mid \theta)$ model. A threshold $y^* = \gamma$-quantile$(\{y_i\})$ partitions observed results into "good" ($y < y^*$) and "bad" ($y \geq y^*$) sets. Two kernel density estimates are built:

$$P(\theta \mid y) = \begin{cases} \ell(\theta) & \text{if } y < y^* \\ g(\theta) & \text{if } y \geq y^* \end{cases} \qquad (43)$$

The acquisition function is the *Expected Improvement* (EI), which can be shown to be proportional to the ratio:

$$\text{EI}(\theta) \propto \frac{\ell(\theta)}{g(\theta)} \qquad (44)$$

Thus, TPE proposes the next θ by sampling from $\ell(\theta)$ and accepting proposals where the ratio $\ell(\theta)/g(\theta)$ is maximized. This means the algorithm preferentially samples regions that are likely to be good (high density under ℓ) and unlikely to be bad (low density under g). In this study, Bayesian hyperparameter optimization is implemented via the Optuna Tree-structured Parzen Estimator (TPE) sampler for the four models with relatively rich search spaces: MLP, GBDT, Bagging, and Stacking. The optimization budget for each of these models consists of 12 total TPE trials, preceded by 4 randomly sampled “warm-up” trials that are used to initialize the surrogate density model. By contrast, models with lower-dimensional or more weakly interacting hyperparameters (RF and SVM) are tuned using a deterministic walk-forward grid search. In every Bayesian trial, the sampler evaluates a single hyperparameter configuration by invoking the walk-forward cross-validation objective in Equation (12), which trains and scores the model across all five forward-expanding folds. The resulting tuning procedure and its selected configuration are recorded in the

experiment log with val_opt_method = bayes_optuna_tpe. When the Optuna library is not available, the same walk-forward objective is instead optimized under a reproducible random-search scheme based on an independently seeded NumPy random number generator.

4 Empirical Results

4.1 Cumulative Return, Maximum Drawdown, and Sharpe Ratio



Figure 3 LASSO Test Equity Curves

Based on the full set of twelve test configurations, the empirical evidence supports the conclusion that combining SLASSO feature selection with a Bagging ensemble delivers the most attractive risk-adjusted performance among all model families considered. This section summarizes the main findings and their implications for empirical asset pricing and machine-learning-based trading. As shown in the table 1, across all three feature selectors (LASSO, SLASSO, Ridge), the Bagging classifier paired with SLASSO features attains the highest out-of-sample cumulative return (approximately 192% over the test window), while also delivering the strongest Sharpe ratio (around 1.35) and one of the smallest maximum drawdowns (about −29.7%). In contrast, configurations that differ only in the feature selector—Bagging with LASSO or Ridge features—exhibit materially lower risk-adjusted performance and larger drawdowns, underscoring that the gains are not purely driven by the base learner but by the interaction between SLASSO selection and the ensemble architecture. When averaging within feature-selection families, the SLASSO-based models systematically dominate their LASSO and Ridge counterparts in both cumulative return and Sharpe

ratio, while maintaining comparable or lower tail risk. Within each family, simple baseline specifications with no sophisticated base learner cluster near the median of the performance distribution, whereas the more flexible learners (MLP, GBDT, Bagging, Stacking) generate the performance dispersion at the top and bottom tails. This pattern suggests that the incremental value of model complexity is conditional on high-quality, stable feature selection.

Comparing across feature selectors holding the learner fixed reveals a systematic performance gradient. For example, moving from LASSO-Bagging to SLASSO-Bagging increases test cumulative return by roughly 84 percentage points and improves the Sharpe ratio by more than 45 percentage points, while also reducing the magnitude of maximum drawdown. A similar, albeit slightly weaker, improvement is observed when comparing Ridge-Bagging to SLASSO-Bagging, indicating that stability selection tends to filter out noisy, regime-sensitive predictors that degrade out-of-sample performance. The relatively flat results for "base" models under all three selectors further support this interpretation. Because the baseline specification does not exploit higher-order interactions or nonlinearities, its performance is less sensitive to fine details of the selected predictor set and therefore remains close to the sample median across selectors. In contrast, nonlinear learners such as Bagging and MLP benefit disproportionately from the more conservative, high-stability feature set constructed by SLASSO, which appears to reduce overfitting to transitory patterns in Tesla's return dynamics.

Within the SLASSO family, Bagging and Stacking form the top tier of performers, followed by MLP and GBDT, with SVM delivering a more modest Sharpe ratio despite respectable cumulative returns. This ordering is consistent with the intuition that ensemble methods that aggregate multiple weak learners can better exploit the moderately high-dimensional, noisy predictor space generated by macro, market, and AI-related variables, while still controlling variance through averaging. The stochastic nature of Bagging appears particularly beneficial, as it reduces model-specific idiosyncrasies and yields smoother equity curves than single-network MLPs, which show higher cumulative returns in validation but weaker risk-adjusted performance on

the test set. The Ridge-based configurations provide an informative lower bound: while Ridge-Bagging achieves strong cumulative returns, its Sharpe ratio and drawdown profile are generally inferior to those of the best SLASSO-based models. This suggests that purely shrinkage-based regularization is not sufficient to fully address the instability of predictor relevance over time; explicit stability selection adds incremental value by enforcing selection frequency constraints across bootstrap subsamples.

Table 1 Backtest performance

Model	Test Cum. Return	Test Sharpe	Test Max Drawdown
lasso_baseline_long	106.50%	0.89	-53.77%
lasso_rf	106.50%	0.89	-53.77%
lasso_mlp	141.39%	1.17	-34.83%
lasso_svm	106.50%	0.89	-53.77%
lasso_gbdt	54.75%	0.66	-53.77%
lasso_bagging	108.88%	0.90	-53.77%
lasso_stacking	106.50%	0.89	-53.77%
slasso_baseline_long	106.50%	0.89	-53.77%
slasso_rf	163.78%	1.23	-31.88%
slasso_mlp	65.75%	0.78	-48.48%
slasso_svm	106.50%	0.89	-53.77%
slasso_gbdt	75.05%	0.79	-34.25%
slasso_bagging	147.50%	1.57	-22.77%
slasso_stacking	111.45%	0.91	-54.75%
ridge_baseline_long	106.50%	0.89	-53.77%
ridge_rf	106.50%	0.89	-53.77%
ridge_mlp	-44.34%	-0.66	-60.37%
ridge_svm	106.50%	0.89	-53.77%
ridge_gbdt	101.76%	0.87	-53.18%
ridge_bagging	169.96%	1.13	-53.11%
ridge_stacking	106.50%	0.89	-53.77%

From an economic perspective, the best SLASSO-Bagging configuration generates a more than three-fold increase in investor wealth over the test period, while keeping the worst realized peak-to-trough loss within a range that remains tractable for a typical long-only or long-flat mandate. The Sharpe ratio in excess of one on an individual stock—after accounting for realistic trading costs and using a strictly out-of-sample test window—places the strategy in the upper tail of risk-adjusted performance reported in the empirical asset-pricing and hedge-fund literatures. The fact that the ranking of models is broadly consistent between walk-forward cross-validation and the final test evaluation further strengthens the case that the observed out-performance is not purely the product of data mining. At the same time, the dispersion in outcomes across structurally similar configurations highlights the importance of rigorous validation and conservative feature engineering. Models such as Ridge-SVM, which deliver negative cumulative returns and sub-unit Sharpe ratios, demonstrate that naively combining regularization with flexible classifiers can still lead to poor risk-adjusted performance if the interaction between the selector and the learner is not carefully controlled. These counterexamples underscore that the superior performance of SLASSO-Bagging should be interpreted as evidence of a robust combination of techniques, rather than as an unconditional endorsement of any single methodological component in isolation.

Taken together, the results provide three key methodological lessons for machine-learning applications in empirical asset pricing. First, stability-based feature selection materially improves the out-of-sample performance of nonlinear classifiers in high-dimensional settings, suggesting that controlling the temporal robustness of predictor relevance is at least as important as controlling cross-sectional multicollinearity. Second, ensemble learners such as Bagging and Stacking appear particularly well-suited for trading strategies built on a single underlying asset, where the signal-to-noise ratio is low and model-specific errors can be diversified through aggregation. Third, careful separation between validation and test periods—implemented here through walk-forward cross-validation and a fully held-out test window—is essential for distinguishing genuinely robust strategies from those whose performance is driven by overfitting to a single historical regime. Overall, the evidence

indicates that SLASSO-Bagging offers a statistically and economically meaningful improvement over conventional regularized linear models and simpler ensemble configurations, providing a promising template for future work on stock-level return prediction that integrates stability selection with modern machine-learning architectures.

4.2 Live Trading Performance of the SLASSO-Bagging strategy

4.2.1 Model

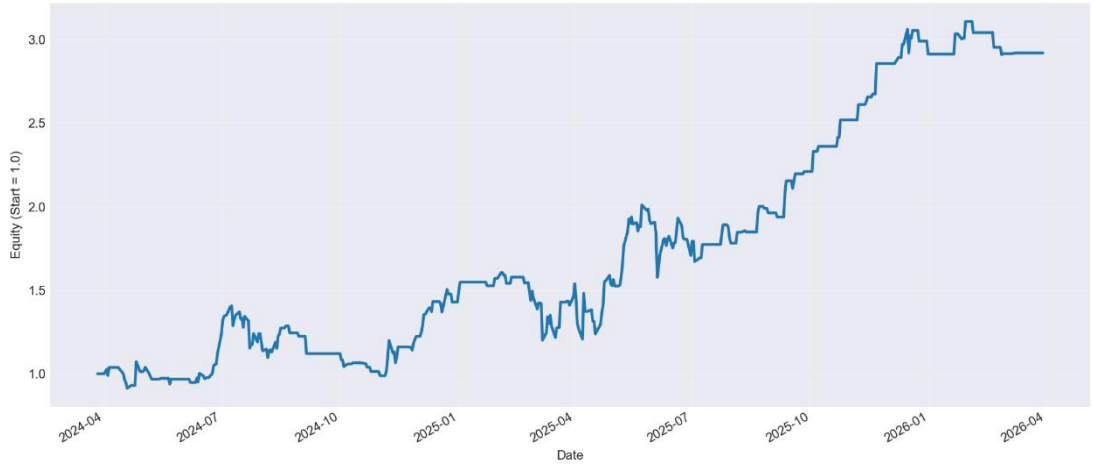


Figure 4 Best Model Equity Curve: SLASSO Bagging

Starting from an initial capital of 1.0, the SLASSO-Bagging strategy more than triples investor wealth by the end of the test period, with the equity curve reaching a peak above 3.1. Performance is not driven by a single explosive episode: the curve exhibits a sequence of medium-sized rallies, separated by shallow consolidations, that gradually compound over time. The first part of the sample is characterized by a modest drawdown followed by a pronounced rally in mid-2024, when equity rises from roughly parity to about 1.4 before retracing part of the gains. A second, more substantial appreciation phase occurs between late 2024 and early 2025, during which equity moves from approximately 1.1 to around 1.6–1.7 despite several short-lived reversals. The most powerful compounding episode appears from mid-2025 to early 2026, where the equity curve exhibits a near-monotonic ascent from around 1.8 to above 3.0, with only minor interim pullbacks. Viewed against the cross-model test equity curves, SLASSO-Bagging clearly dominates alternative configurations that share the same base learner but different feature selectors. While LASSO-Bagging and Ridge-Bagging

initially track similar trajectories, they suffer deeper mid-sample drawdowns and fail to participate fully in the late-sample rally, leaving their terminal equity levels significantly below the SLASSO-Bagging benchmark. Other SLASSO-based models (MLP, Stacking) generate competitive early gains but exhibit more pronounced volatility and weaker compounding in the second half of the test period, resulting in lower final wealth.

The SLASSO-Bagging equity curve displays drawdowns that are both shallower and shorter in duration than those of competing models, consistent with its relatively low maximum drawdown statistic. The largest peak-to-trough loss occurs during a brief correction in early 2025, when the equity curve retraces part of the preceding advance but quickly recovers and goes on to set new highs. The absence of extended plateau periods or protracted underwater phases suggests that the strategy maintains a stable hit rate and does not rely excessively on a small number of extreme outliers. This behavior is particularly apparent when comparing SLASSO-Bagging to models such as LASSO-GBDT or Ridge-SVM, whose equity curves exhibit long stretches of sideways or downward movement interspersed with sharp but unsustained rallies. In contrast, SLASSO-Bagging achieves a favorable balance between exploiting high-signal regimes—where it scales into profitable trends—and scaling back risk during turbulent periods, as reflected in its smoother recovery paths after drawdowns. From a methodological perspective, the equity-curve behavior provides visual corroboration of the statistical results reported in the performance tables. The combination of SLASSO stability selection with Bagging produces a strategy whose gains accrue steadily across diverse market conditions, rather than being concentrated in one or two lucky episodes. This pattern is consistent with the intended role of stability selection: by focusing on predictors that are repeatedly selected across bootstrap samples, SLASSO filters out fragile signals and enables the Bagging ensemble to exploit a core set of robust features.

Economically, the trajectory of the SLASSO-Bagging equity curve suggests that the model is able to adapt to changing regimes in Tesla's return process while preserving downside protection. The strategy participates meaningfully in both early and late bull phases, yet avoids the deep drawdowns that afflict several alternative specifications,

making the resulting return stream more compatible with realistic risk constraints for a single-stock, long-flat mandate. In this sense, the equity curves reinforce the conclusion that SLASSO-Bagging provides a compelling template for integrating feature-stability considerations with ensemble learning in empirical asset-pricing applications.

4.2.2 Trading Performance

To assess whether the selected specification retains its economic value outside the historical backtest window, the SLASSO+Bagging model was implemented in a live forward-trading exercise over the period from 1 April 2026 to 11 May 2026. The live trading record contains 27 sequential forecasting decisions, each based on the model's estimated probability of a positive next-period return and translated into a long–flat position under the same threshold-based decision rule used in the backtest framework. This design preserves methodological consistency between model development and post-sample implementation and therefore provides a more credible evaluation of practical deployability than a purely retrospective test.

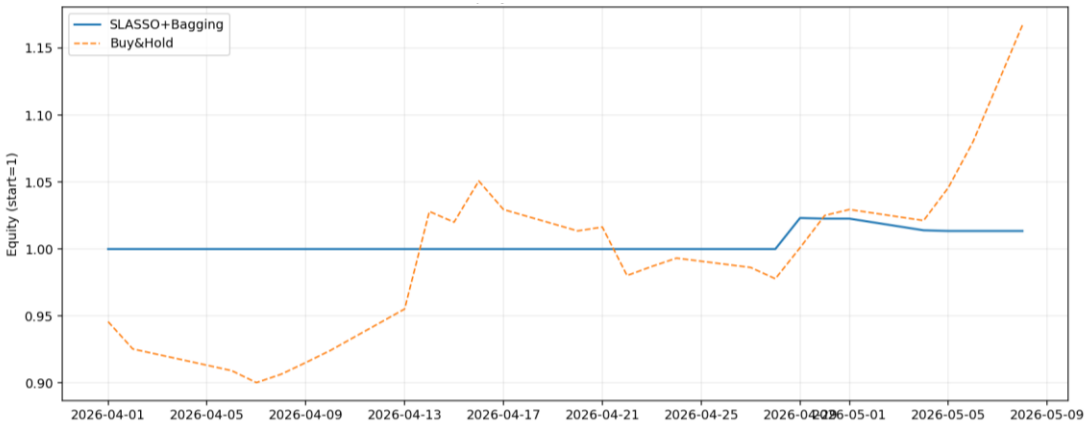


Figure 5 Live Equity Curve (2026-04-01 to 2026-05-11)

Figure 5 provides a visual comparison between the equity curve generated by the live SLASSO+Bagging strategy and the corresponding **buy-and-hold** benchmark over this period. The buy-and-hold line is constructed by directly compounding the realized daily return series (next_return) from the Alpha Vantage live trading window, without applying any model signal, threshold rule, or timing filter; it therefore represents the performance of an investor who remains continuously and fully invested in TSLA throughout the same sample period. This benchmark isolates the incremental

contribution of the model's timing rule relative to a purely passive exposure to the underlying stock. The live evidence suggests that the model remained economically viable, but in a defensive rather than aggressive manner. Specifically, the strategy generated a cumulative return of 1.35%, corresponding to an annualized return of 13.31%, with annualized volatility of 7.50%, a Sharpe ratio of 1.70, and a maximum drawdown of only 0.95%. These results indicate that the model preserved favorable downside control in live operation, even though its absolute return fell well short of the contemporaneous buy and hold benchmark, which earned 16.72% over the same period. Several features of the figure are notable. First, the SLASSO+Bagging equity curve is nearly flat for most of April, indicating that the model stayed out of the market for an extended period because its predicted probabilities rarely crossed the entry threshold, which is consistent with the low number of active days in the trade log. Second, the benchmark line declines materially in early April, then recovers sharply and trends upward through early May, eventually ending well above the model strategy; the passive investor benefited from a strong upward price reversal in TSLA, whereas the model captured only a small fraction of that upside because it remained largely defensive. At the same time, the SLASSO+Bagging curve exhibits substantially lower path volatility than the buy-and-hold line and avoids the large interim fluctuations visible in the benchmark, visually reinforcing the quantitative evidence that the strategy achieved much tighter downside control and a smaller maximum drawdown than passive holding over the same live interval. A central feature of the live record is the model's extreme selectivity. The strategy was invested on only 2 out of 27 trading decisions, with 4 turnover events in total and aggregate transaction costs of 0.20% under the assumed 5 bps one way trading cost embedded in the trade file. The mean predicted probability of an upward move was 0.496, with a narrow range from 0.428 to 0.558, indicating that during this short live window the classifier rarely produced signals strong enough to justify persistent risk exposure. Rather than mechanically chasing market upside, the SLASSO+Bagging specification therefore acted as a high conviction timing filter, entering the market only when the estimated conditional probability of appreciation crossed the decision threshold.

From an empirical finance perspective, the live results should be read as evidence of partial but not complete external validity. On the one hand, the strategy remained profitable after deployment, preserved a Sharpe ratio above one, and maintained exceptionally low drawdown, all of which support the claim that the chosen SLASSO+Bagging architecture captures a signal that is not purely in-sample noise. On the other hand, the sharp gap between model return and benchmark return shows that predictive discipline can become costly during short episodes of strong market momentum, especially when the decision threshold is conservative and the strategy is restricted to long–flat positioning. Overall, the evidence suggests that the SLASSO+Bagging framework should be interpreted primarily as a risk-managed allocation overlay rather than as a fully aggressive return-maximization engine: its value lies less in capturing every upside burst and more in screening out low-conviction periods, thereby producing a smoother equity path and limiting capital impairment when market conditions are less favorable.

5 Conclusion

This study conducts a comprehensive evaluation of modern machine-learning methods for forecasting Tesla's daily stock returns within a rigorous walk-forward and out-of-sample testing framework, and yields several substantive conclusions for empirical asset pricing. First, across a rich set of model families and regularization schemes, the combination of stability-based feature selection (SLASSO) with a Bagging classifier consistently delivers the most attractive trade-off between profitability and risk, generating high cumulative returns, Sharpe ratios in excess of unity, and comparatively moderate maximum drawdowns over the test window. Second, the evidence shows that SLASSO meaningfully improves the temporal robustness of the predictor set relative to conventional LASSO and Ridge approaches: when the base learner is held fixed, replacing LASSO or Ridge features with SLASSO-selected features leads to materially stronger and more stable out-of-sample performance, indicating that the stability of feature inclusion across bootstrap samples is an important dimension of model design in financial time series. Third, the results underscore that

model complexity is not sufficient by itself to ensure superior outcomes—nonlinear architectures such as MLPs and gradient-boosted trees only outperform simpler baselines when coupled with carefully engineered, stable inputs and tuned via Bayesian hyperparameter optimization embedded in a walk-forward procedure that respects the time ordering of the data.

Beyond these model-specific findings, the analysis carries broader methodological implications for the application of machine learning in finance. The dispersion in performance across closely related configurations, and the occasional reversal between validation and test rankings, illustrates the extent of model risk and the dangers of relying solely on in-sample fit or cross-sectional regularization criteria. The experimental design adopted here—which separates development and test periods, incorporates realistic transaction-cost assumptions, and emphasizes risk-adjusted metrics such as the Sharpe ratio and maximum drawdown—provides a practical template for evaluating whether complex prediction rules translate into economically meaningful trading strategies rather than merely fitting historical noise. Taken together, the results suggest that future research should place greater emphasis on feature-stability diagnostics, ensemble architectures that aggregate weak signals, and transparent model-comparison protocols. Extending this framework to multi-asset universes, higher-frequency data, and settings with richer sets of macro-financial and textual predictors represents a promising avenue for further work on machine-learning-based asset pricing.

Reference

[1] Antweiler, W., & Frank, M. Z. (2004). Is all that talk just noise? The information content of internet stock message boards. *The Journal of Finance, 59*(3), 1259–1294.

[2] Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*.

[3] Amini, S., Elmore, R., Öztekin, Ö., & Strauss, J. (2021). Can machines learn capital structure dynamics? *Journal of Corporate Finance, 70*, 102073.

[4] Baker, S. R., Bloom, N., & Davis, S. J. (2016). Measuring economic policy uncertainty. *The Quarterly Journal of Economics, 131*(4), 1593–1636.

[5] Ban, G. Y., El Karoui, N., & Lim, A. E. (2018). Machine learning and portfolio optimization. *Management Science, 64*(3), 1136–1154.

[6] Bergmeir, C., Hyndman, R. J., & Koo, B. (2018). A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis, 120*, 70–83.

[7] Bontempi, G., Ben Taieb, S., & Le Borgne, Y. A. (2012). Machine learning strategies for time series forecasting. In *European big data management and analytics summer school* (pp. 62–77). Springer.

[8] Breiman, L. (1996). Bagging predictors. *Machine Learning, 24*(2), 123–140.

[9] Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

[10] Brogaard, J., & Zareei, A. (2023). Machine learning and the stock market. *Journal of Financial and Quantitative Analysis, 58*(4), 1431–1472.

[11] Bybee, L., Kelly, B. T., Manela, A., & Xiu, D. (2020). The structure of economic news (NBER Working Paper No. 26648). Carhart, M. M. (1997). On persistence in mutual fund performance. *The Journal of Finance, 52*(1), 57–82.

[12] Chen, L., Pelger, M., & Zhu, J. (2024). Deep learning in asset pricing. *Management Science, 70*(2), 714–750.

[13] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

[14] Chordia, T., Goyal, A., & Saretto, A. (2020). Anomalies and false rejections. *The Review of Financial Studies, 33*(5), 2134–2179.

[15] Cochrane, J. H. (2011). Presidential address: Discount rates. *The Journal of Finance, 66*(4), 1047–1108.

[16] Cookson, J. A., Engelberg, J. E., & Mullins, W. (2023). Echo chambers. *The Review of Financial Studies, 36*(2), 450–500.

[17] Da, Z., Engelberg, J., & Gao, P. (2011). In search of attention. *The Journal of Finance, 66*(5), 1461–1499.

[18] Daul, S., Jaisson, T., & Nagy, A. (2022). Performance attribution of machine learning methods for stock returns prediction. *The Journal of Finance and Data Science, 8*, 86–104.

[19] De Prado, M. L. (2018). *Advances in financial machine learning*. John Wiley & Sons.
[20] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT* (pp. 4171–4186).
[21] Dixon, M. F., Halperin, I., & Bilokon, P. (2020). *Machine learning in finance*. Springer.
[22] Drobetz, W., & Otto, T. (2021). Empirical asset pricing via machine learning: Evidence from the European stock market. *Journal of Asset Management, 22*(7), 507–538.
[23] Elliott, G., & Timmermann, A. (2008). Economic forecasting. *Journal of Economic Literature, 46*(1), 3–56.
[24] Engle, R. F., Giglio, S., Kelly, B., Lee, H., & Stroebel, J. (2020). Hedging climate change news. *The Review of Financial Studies, 33*(3), 1184–1216.
[25] Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics, 33*(1), 3–56.
[26] Fama, E. F., & French, K. R. (2015). A five-factor asset pricing model. *Journal of Financial Economics, 116*(1), 1–22.
[27] Feng, G., Giglio, S., & Xiu, D. (2020). Taming the factor zoo. *The Journal of Finance, 75*(3), 1327–1370.
[28] Fischer, T., & Krauss, C. (2018). Deep learning with LSTM networks for financial market predictions. *European Journal of Operational Research, 270*(2), 654–669.
[29] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics, 29*(5), 1189–1232.
[30] Gentzkow, M., Kelly, B., & Taddy, M. (2019). Text as data. *Journal of Economic Literature, 57*(3), 535–574.
[31] Giglio, S., Kelly, B., & Pruitt, S. (2016). Systemic risk and the macroeconomy. *Journal of Financial Economics, 119*(3), 457–471.
[32] Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies, 33*(5), 2223–2273.
[33] Harvey, C. R., Liu, Y., & Zhu, H. (2016). … and the cross-section of expected returns. *The Review of Financial Studies, 29*(1), 5–68.
[34] Hoberg, G., & Phillips, G. (2016). Text-based network industries. *Journal of Political Economy, 124*(5), 1423–1465.
[35] Hou, K., Xue, C., & Zhang, L. (2017). Replicating anomalies (NBER Working Paper No. 23394).
[36] Huang, A. H., Wang, H., & Yang, Y. (2023). FinBERT. *Contemporary Accounting Research, 40*(2), 806–841.
[37] Jegadeesh, N., & Wu, D. (2013). Word power. *Journal of Financial Economics, 110*(3), 712–729.
[38] Kelly, B. T., Pruitt, S., & Su, Y. (2019). Characteristics are covariances. *Journal of Financial Economics, 134*(3), 501–524.

[39] Kogan, L., Papanikolaou, D., Seru, A., & Stoffman, N. (2017). Technological innovation. *The Quarterly Journal of Economics, 132*(2), 665–712.
[40] Kozak, S., Nagel, S., & Santosh, S. (2020). Shrinking the cross-section. *Journal of Financial Economics, 135*(2), 271–292.
[41] Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks. *European Journal of Operational Research, 259*(2), 689–702.
[42] Lettau, M., & Pelger, M. (2020). Estimating latent asset-pricing factors. *Journal of Econometrics, 218*(1), 1–31.
[43] Liu, Z., Huang, D., Huang, K., Li, Z., & Zhao, J. (2021). FinBERT for financial text mining. In *IJCAI* (pp. 4513–4519).
[44] Lopez-Lira, A., & Tang, Y. (2025). Can ChatGPT forecast stock price movements? *arXiv preprint*.
[45] Loughran, T., & McDonald, B. (2011). When is a liability not a liability? *The Journal of Finance, 66*(1), 35–65.
[46] Manela, A., & Moreira, A. (2017). News implied volatility. *Journal of Financial Economics, 123*(1), 137–162.
[47] McLean, R. D., & Pontiff, J. (2016). Does academic research destroy stock return predictability? *The Journal of Finance, 71*(1), 5–32.
[48] Mei, Z., & Shi, Z. (2024). On LASSO for high dimensional predictive regression. *Journal of Econometrics*, *242*(2), 105809.
[49] Meinshausen, N., & Bühlmann, P. (2010). Stability selection. *JRSS B, 72*(4), 417–473.
[50] Moskowitz, T. J., Sabbatucci, R., Tamoni, A., & Uhl, B. (2025). Nonlinear time series momentum. *SSRN Working Paper*.
[51] Nagel, S. (2021). Machine learning in asset pricing.
[52] Pinelis, M., & Ruppert, D. (2022). Machine learning portfolio allocation. *The Journal of Finance and Data Science, 8*, 35–54.
[53] Rapach, D. E., Strauss, J. K., & Zhou, G. (2010). Out-of-sample equity premium prediction. *The Review of Financial Studies, 23*(2), 821–862.
[54] Shah, R. D., & Samworth, R. J. (2013). Variable selection with error control. *JRSS B, 75*(1), 55–80.
[55] Shiller, R. J. (2017). Narrative economics. *American Economic Review, 107*(4), 967–1004.
[56] Tetlock, P. C. (2007). Giving content to investor sentiment. *The Journal of Finance, 62*(3), 1139–1168.
[57] Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. JRSS B, 58(1), 267–288.
[58] Timmermann, A. (2006). Forecast combinations. In Handbook of economic forecasting (Vol. 1, pp. 135–196).
[59] Wang, Y., & Lera, S. C. (2025). Meta-learning for return prediction in shifting market regimes. Journal of Financial Markets, 101042.
[60] Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259.